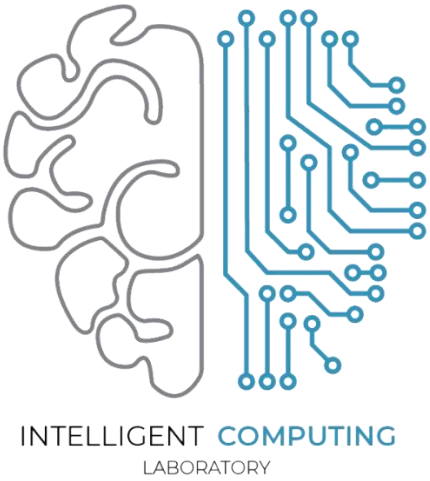
LAPORAN TEORI

MACHINE LEARNING



NAMA : Fathur Rasyid Irimawa

NIM : 202431036

KELAS : C

DOSEN : Dr. Efy Yosrita, S.Si., M.Kom

NO.PC : PC 33

ASISTEN : 1. Muhammad Raka Ilham

2. Sharira Maharani

3. Muhammad Hanif

4. Muhammad Fachrydzi

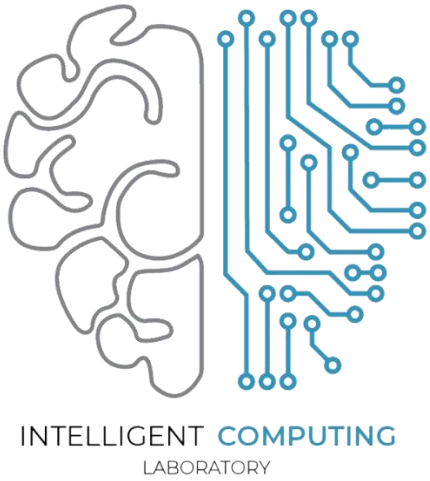
INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

LAPORAN PRAKTIKUM

MACHINE LEARNING



NAMA : Fathur Rasyid Irimawa

NIM : 202431036

KELAS : C

DOSEN : Dr. Efy Yosrita, S.Si., M.Kom

NO.PC : PC 33

ASISTEN : 1. Muhammad Raka Ilham

2. Sharira Maharani

3. Muhammad Hanif

4. Muhammad Fachrydzi

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

1. EDA
   - Memanggil library

     Kode: import pandas as pd
     import matplotlib.pyplot as plt

     Output: output tidak ada karna hanya memanggil library

     Penjelasan: memanggil pustaka pandas untuk melakukan manipulasi, proses, dan menganalisis data yang memiliki struktur tabel, dan matplotlib dipanggil agar bisa melakukan visualisasi data seperti grafik, diagram dan lain-lain.
   - Membaca dataset dan menampilkan data

     Kode: df = pd.read_csv('tennis_courts (EDA).csv')
     Print(df.head(5))

     Output:

	index	name	address	city	state	zip_code	type	count	clay	wall	grass	indoor	lights	proshop	latitude	longitude
0	0	Adelphi Road	washington	dc	20782.0	Public	6	False	True	False	False	False	False	38.974351	-76.952112	{"clay"=>"0", "type"=>"Public", "wall"=>"1", "...
1	1	Brandywood Circle	orlando	fl	32792.0	Homeowners Community	1	False	False	False	False	True	False	28.590956	-81.285224	{"clay"=>"0", "type"=>"Homeowners Community", ...
2	2	Greenlee Drive	austin-san marcos	tx	78628.0	Public	8	False	True	False	False	True	False	30.615173	-97.700794	{"clay"=>"0", "type"=>"Public", "wall"=>"1", "...
3	3	McKee Street	minneapolis-st. paul	mn	55121.0	Public	1	False	False	False	False	False	False	44.852096	-93.142100	{"clay"=>"0", "type"=>"Public", "wall"=>"0", "...
4	4	Avenue North	minneapolis-st. paul	mn	55374.0	Public	6	False	False	False	False	False	False	45.207841	-93.544174	{"clay"=>"0", "type"=>"Public", "wall"=>"0", "...

     Penjelasan: kode diatas digunakan untuk membaca file dengan format csv dengan pustaka pandas kemudian disimpan ke dalam bentuk tabel, setelah disimpan data kemudian ditampilkan sebanyak 5 data
   - Melihat informasi data

     Kode: df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29959 entries, 0 to 29958
Data columns (total 16 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   index      29959 non-null  int64
 1   name       29959 non-null  object
 2   address    29959 non-null  object
 3   city       29959 non-null  object
 4   state      29958 non-null  float64
 5   zip_code   29959 non-null  object
 6   type       29959 non-null  int64
 7   count      29959 non-null  bool
 8   clay       29959 non-null  bool
 9   wall       29959 non-null  bool
 10  grass      29959 non-null  bool
 11  indoor     29959 non-null  bool
 12  lights     29959 non-null  bool
 13  proshop    29959 non-null  float64
 14  latitude   29959 non-null  float64
 15  longitude  29959 non-null  object
dtypes: bool(6), float64(3), int64(2), object(5)
memory usage: 2.5+ MB
```

     Output:

     Penjelasan: menampilkan ringkasan informasi dari df. Dan informasi nya yaitu jumlah baris dan kolom, nama dan tipe data, jumlah data, dan estimasi jumlah memori yang terpakai.
   - Melihat deskripsi statistik

     Kode: print(df.describe(include="all"))

	index	name	address	city	state	zip_code
count	29959.000000	29959	29959	29959	29958.000000	29959
unique	NaN	22313	54	30	NaN	2
top	NaN	West Street	new york	ca	NaN	Public
freq	NaN	212	2663	3864	NaN	16227
mean	14979.000000	NaN	NaN	NaN	47890.646806	NaN
std	8648.562694	NaN	NaN	NaN	29921.742686	NaN
min	0.000000	NaN	NaN	NaN	1001.000000	NaN
25%	7489.500000	NaN	NaN	NaN	27103.000000	NaN
50%	14979.000000	NaN	NaN	NaN	38128.000000	NaN
75%	22468.500000	NaN	NaN	NaN	77379.000000	NaN
max	29958.000000	NaN	NaN	NaN	98685.000000	NaN

	type	count	clay	wall	grass	indoor	lights	proshop
count	29959.000000	29959	29959	29959	29959	29959	29959	29959.000000
unique	NaN	2	2	2	2	2	2	NaN
top	NaN	False	False	False	False	False	False	NaN
freq	NaN	29481	25198	29949	29873	16208	29796	NaN
mean	3.003905	NaN	NaN	NaN	NaN	NaN	NaN	36.741224
std	2.497830	NaN	NaN	NaN	NaN	NaN	NaN	5.361387
min	0.000000	NaN	NaN	NaN	NaN	NaN	NaN	25.448632
25%	2.000000	NaN	NaN	NaN	NaN	NaN	NaN	33.525299
50%	2.000000	NaN	NaN	NaN	NaN	NaN	NaN	37.779415
75%	4.000000	NaN	NaN	NaN	NaN	NaN	NaN	40.902716
max	41.000000	NaN	NaN	NaN	NaN	NaN	NaN	48.301367

	latitude	longitude
count	29959.000000	29959
unique	NaN	509
top	NaN	{"clay"=>"0", "type"=>"Homeowners Community", ...
freq	NaN	3519
mean	-90.207115	NaN
std	15.796396	NaN
min	-123.394972	NaN
25%	-97.083823	NaN
50%	-84.287560	NaN
75%	-78.961160	NaN
max	-70.377785	NaN

Output:

Penjelasan: kode digunakan untuk menampilkan ringkasan data statistik dari seluruh kolom di dalam tabel df. (DataFrame), dan parameter include all digunakan untuk memastikan analisis dilakukan tidak pada kolom angka saja namun juga pada kolom teks.

- Menghitung data kosong di tiap kolom

  Kode: print(df.isna().sum())

```
index        0
name         0
address      0
city         0
state        1
zip_code     0
type         0
count        0
clay         0
wall         0
grass        0
indoor       0
lights       0
proshop      0
latitude     0
longitude    0
dtype: int64
```

Output:

Penjelasan: kode berfungsi untuk menghitung dan menampilkan jumlah data yang kosong (yang formatnya NaN) pada setiap kolom di dalam dataframe. Dimana proses bekerja secara dua tahap, tahap pertama yaitu isna untuk mengecek setiap sel dalam tabel apakah nilainya kosong atau tidak (true jika nilai kosong), tahap kedua menjumlahkan nilai True untuk setiap masing-masing kolom

- Melihat isi kolom kategorik

  Kode:
```
cat_cols = df.select_dtypes(include=['object']).columns
for col in cat_cols:
    print(f"Nilai unik kolom '{col}': {df[col].unique()}")
```

```
Nilai unik kolom 'name': ['Adelphi Road' 'Brandywood Circle' 'Greenlee Driv
e' ... 'Bel Air Drive'
 'La Mirage Circle' 'Ledwicke Street']
Nilai unik kolom 'address': ['washington' 'orlando' 'austin-san marcos' 'min
neapolis-st. paul'
 'west palm beach - boca raton' 'new york' 'chicago' 'denver' 'milwaukee'
 'atlanta' 'detroit' 'boston' 'baltimore' 'memphis' 'albany'
 'dayton-springfield' 'oklahoma city' 'st. louis'
 'san francisco - oakland - san jose' 'fresno' 'dallas - fort worth'
 'houston - galveston - brazoria' 'tampa-st. petersburg-clearwater'
 'san antonio' 'richmond-petersburg' 'cleveland' 'cincinnati'
 'raleigh-durham-chapel hill' 'grand rapids' 'indianapolis' 'los angeles'
 'philadelphia' 'seattle' 'pittsburgh' 'portland' 'miami-fort lauderdale'
 'nashville' 'wilmington-newark' 'hartford' 'louisville' 'kansas city'
 'phoenix' 'sacramento-yolo' 'jacksonville' 'rochester'
 'buffalo-niagara falls' 'norfolk-virginia beach' 'columbus' 'providence'
 'greensboro-winston-salem' 'charlotte' 'san diego' 'salt lake city'
 'las vegas']
```

Output: `Nilai unik kolom 'city': ['dc' 'fl' 'tx' 'mn' 'ny' 'il' 'co' 'wi' 'ga' 'mi'` (Data ditampilkan sebagian saja karna sangat banyak)

Penjelasan: kode berfungsi untuk menampilkan semua nilai unique atau nilai yang berbeda dari setiap kolom teks di dalam dataframe. Baris kode select_dtypes... digunakan untuk mencari dan mengumpulkan semua nama kolom yang bertipe data objek, dan perulangan for mengecek setiap kolom secara satu persatu lalu mencetak daftar nilai unique yang ada di dalam perintah kode unique()

- Mengisi data kosong dengan modus dan mean

Kode:
```
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
cat_cols = df.select_dtypes(include=['object', 'bool']).columns
for col in num_cols:
    df[col].fillna(df[col].mean(), inplace=True)
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
print("Data kosong setelah diisi:")
print(df.isna().sum())
```

```
Data kosong setelah diisi:
index        0
name         0
address      0
city         0
state        0
zip_code     0
type         0
count        0
clay         0
wall         0
grass        0
indoor       0
lights       0
proshop      0
latitude     0
longitude    0
dtype: int64
```

Output:

Penjelasan: kode berfungsi untuk mengisi data kosong. Kolom angka diisi nilai rata-rata, kolomteks dan boolean diisi dengan nilai yang paling sering muncul (modus), kemudian proses terakhir kode mengecek sisa data yang kosong

- Menghitung data duplikat

Kode:
```
print("Jumlah duplikat:", df.duplicated().sum())
```

Output: `Jumlah duplikat: 0`

Penjelasan: kode digunakan untuk menghitung dan menampilkan total baris data yang terduplikasi di dalam dataframe

- Menghapus data duplikat

Kode:
```
df = df.drop_duplicates()
df.info()
```

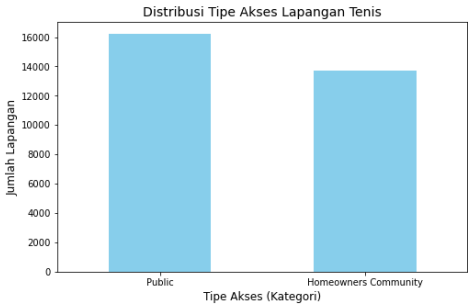
```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 29959 entries, 0 to 29958
Data columns (total 16 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   index      29959 non-null  int64
 1   name       29959 non-null  object
 2   address    29959 non-null  object
 3   city       29959 non-null  object
 4   state      29959 non-null  float64
 5   zip_code   29959 non-null  object
 6   type       29959 non-null  int64
 7   count      29959 non-null  bool
 8   clay       29959 non-null  bool
 9   wall       29959 non-null  bool
 10  grass      29959 non-null  bool
 11  indoor     29959 non-null  bool
 12  lights     29959 non-null  bool
 13  proshop    29959 non-null  float64
 14  latitude   29959 non-null  float64
 15  longitude  29959 non-null  object
dtypes: bool(6), float64(3), int64(2), object(5)
memory usage: 2.7+ MB
```

Output:

Penjelasan: kode berfungsi menghapus semua baris data yang terduplikasi dari dataframe, lalu menampilkan kembali ringkasan informasinya untuk memverifikasi perubahan struktur setelah data dibersihkan.

- Visualisasi Data

Kode:
```
plt.figure(figsize=(8,5))
df['zip_code'].value_counts().plot(kind='bar', color='skyblue')
plt.title("Distribusi Tipe Akses Lapangan Tenis", fontsize=14)
plt.xlabel("Tipe Akses (Kategori)", fontsize=12)
plt.ylabel("Jumlah Lapangan", fontsize=12)
plt.xticks(rotation=0)
plt.show()
```



Output:

Penjelasan: Kode ini berfungsi untuk membuat dan menampilkan grafik batang yang menunjukkan distribusi jumlah data. Kode mengatur ukuran gambar, warna grafik, menambahkan judul dan label sumbu, serta membuat teks sumbu X mendatar agar mudah dibaca.

2. Scrapping

- Mengunduh Request

Kode: !pip install requests beautifulsoup4 pandas lxml

```
Requirement already satisfied: requests in d:\anaconda\lib\site-packages (2.
25.1)
Requirement already satisfied: beautifulsoup4 in d:\anaconda\lib\site-packag
es (4.9.3)
Requirement already satisfied: pandas in d:\anaconda\lib\site-packages (1.2.
4)
Requirement already satisfied: lxml in d:\anaconda\lib\site-packages (4.6.3)
Requirement already satisfied: soupsieve>1.2 in d:\anaconda\lib\site-package
s (from beautifulsoup4) (2.2.1)
Requirement already satisfied: pytz>=2017.3 in d:\anaconda\lib\site-packages
(from pandas) (2021.1)
Requirement already satisfied: python-dateutil>=2.7.3 in d:\anaconda\lib\sit
e-packages (from pandas) (2.8.1)
Requirement already satisfied: numpy>=1.16.5 in d:\anaconda\lib\site-package
s (from pandas) (1.20.1)
Requirement already satisfied: six>=1.5 in d:\anaconda\lib\site-packages (fr
om python-dateutil>=2.7.3->pandas) (1.15.0)
Requirement already satisfied: idna<3,>=2.5 in d:\anaconda\lib\site-packages
(from requests) (2.10)
```

Output:

Penjelasan: menginstall request agar bisa ekstrak data dari web (scraping)

- Memanggil Library

  Kode:
  ```
  import requests
  from bs4 import BeautifulSoup
  import pandas as pd
  import csv
  import re
  ```

  Output: tidak ada

  Penjelasan: Mengimpor berbagai pustaka Python untuk web scraping (requests, BeautifulSoup), pengolahan data (pandas, csv), dan pencarian pola teks (re).

- Mengambil halaman web

  Kode:
  ```
  url = "https://en.wikipedia.org/wiki/List_of_countries_by_external_debt" headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)'} response = requests.get(url, headers=headers)
  print(f"Status Code: {response.status_code}") if response.status_code == 200: print("Berhasil terhubung ke Wikipedia!\n")
  ```

  Output:
  ```
  Status Code: 200
  Berhasil terhubung ke Wikipedia!
  ```

  Penjelasan: Kode berfungsi untuk mengakses halaman Wikipedia menggunakan request khusus. Jika koneksi berhasil dengan status 200, akan mencetak pesan sukses.

- Html parser

  Kode: `soup = BeautifulSoup(response.text, 'html.parser')`

  Output: tidak ada

  Penjelasan: Kode berfungsi untuk mem-parsing teks HTML web menjadi objek yang terstruktur.

- Mencari Data

  Kode:
  ```
  tables = soup.find_all('table', class_='wikitable') target_table = tables[0]
  rows = target_table.find_all('tr')[1:] print(f"Jumlah blok data (baris) ditemukan: {len(rows)}") if len(rows) >= 200: print("Syarat minimal 200 data terpenuhi!\n")
  ```

  Output:
  ```
  Jumlah blok data (baris) ditemukan: 209
  Syarat minimal 200 data terpenuhi!
  ```

  Penjelasan: Menyeleksi tabel pertama, mengambil semua baris data tanpa header, lalu mengecek apakah jumlah data mencapai 200 baris minimalnya.

- Mengambil atribut dari setiap blok

  Kode:
  ```
  result = [] for row in rows: cols = row.find_all(['td', 'th'])
  if len(cols) >= 5:
      country = cols[0].get_text(strip=True).split('[')[0]
      external_debt = cols[1].get_text(strip=True).split('[')[0]
      per_capita = cols[3].get_text(strip=True).split('[')[0]
      gdp_percentage = cols[4].get_text(strip=True).split('[')[0]
  result.append({
      'Country': country,
      'External_Debt': external_debt,
      'Per_Capita': per_capita,
      'GDP_Percentage': gdp_percentage
  })
  ```

Output: tidak ada

Penjelasan: Mengekstrak, membersihkan teks dari simbol referensi, dan menyimpan data (negara, utang, per kapita, dan persentase PDB) dari baris tabel ke dalam list result.

- Menampilkan 5 data pertama dari proses scraping

Kode:
```
print("Menampilkan 5 data pertama dari list hasil scraping:")
for i in range(5):
    print(result[i])
```

Output:
```
Menampilkan 5 data pertama dari list hasil scraping:
{'Country': 'United States', 'External_Debt': '40,598.133', 'Per_Capita': '93.2
5', 'GDP_Percentage': '21.59'}
{'Country': 'United Kingdom', 'External_Debt': '11,355', 'Per_Capita': '278.1
8', 'GDP_Percentage': '5.91'}
{'Country': 'France', 'External_Debt': '9,249.4815', 'Per_Capita': '257.21', 'G
DP_Percentage': '57.85'}
{'Country': 'Germany', 'External_Debt': '8,139.6669', 'Per_Capita': '149.27',
'GDP_Percentage': '46.71'}
{'Country': 'Netherlands', 'External_Debt': '4,877.3109', 'Per_Capita': '336.4
3', 'GDP_Percentage': '100.17'}
```

Penjelasan: Mencetak pesan pengantar, lalu menggunakan perulangan untuk menampilkan lima data pertama dari list result hasil scraping.

- Membuat file .csv

Kode:
```
csv_filename = "data_utang_negara.csv" with open(csv_filename, mode='w', newline='', encoding='utf-8') as f: writer = csv.DictWriter(f, fieldnames=['Country', 'External_Debt', 'Per_Capita', 'GDP_Percentage']) writer.writeheader() for row_data in result: writer.writerow(row_data) print(f"\nData mentah berhasil disimpan ke {csv_filename}")
```

Output:
```
Data mentah berhasil disimpan ke data_utang_negara.csv
```

Penjelasan: Membuat file "data_utang_negara.csv", menulis judul kolom, lalu menyimpan seluruh baris data dari variabel result ke dalam file berformat CSV, dan mencetak pesan sukses.

- Membaca file .csv dan menampilkan data

Kode:
```
df = pd.read_csv(csv_filename) print("\nData dibaca menggunakan Pandas (Tipe data awal masih berupa Object/Teks) 😊 print(df.head())
```

Output:
```
Data dibaca menggunakan Pandas (Tipe data awal masih berupa Object/Teks):
          Country External_Debt Per_Capita  GDP_Percentage
0   United States    40,598.133      93.25           21.59
1  United Kingdom        11,355     278.18            5.91
2          France     9,249.4815     257.21           57.85
3         Germany     8,139.6669     149.27           46.71
4     Netherlands     4,877.3109     336.43          100.17
```

Penjelasan: Membaca file CSV ke dalam dataframe pandas, mencetak pesan pengantar, lalu menampilkan lima baris pertama untuk memeriksa data mentah yang masih berbentuk teks sebelum diolah.

- Memastikan kolom angka bertipe int/float

Kode:
```
def clean_to_numeric(text): if pd.isna(text): return None text = str(text).replace(',', '').strip() num = re.sub(r'[^\d.]', '', text) if num == '': return None return float(num)
df['External_Debt'] = df['External_Debt'].apply(clean_to_numeric)
df['Per_Capita'] = df['Per_Capita'].apply(clean_to_numeric)
df['GDP_Percentage'] = df['GDP_Percentage'].apply(clean_to_numeric)
df = df.dropna()
```

Output: tidak ada

Penjelasan: Mendefinisikan fungsi untuk membersihkan teks dari karakter non-angka dan mengubahnya menjadi angka (float). Fungsi tersebut diterapkan pada tiga kolom, lalu baris data yang kosong dihapus.

- Membuat kolom kategori logis berdasarkan persentase GDP

Kode: Proses tambahan untuk soal nomor 4 decision tree
df['Risk_Category'] = df['GDP_Percentage'].apply(lambda x: 'Risiko Tinggi' if x > 60 else 'Risiko Rendah') df.to_csv("data_utang_negara_clean.csv", index=False)
print("\nData setelah dibersihkan dan diubah menjadi tipe angka:") print(df.dtypes) print("\nTampilan Data Bersih Siap Pakai:") print(df.head())

Output:
```
Data setelah dibersihkan dan diubah menjadi tipe angka:
Country           object
External_Debt     float64
Per_Capita        float64
GDP_Percentage    float64
Risk_Category      object
dtype: object

Tampilan Data Bersih Siap Pakai:
         Country  External_Debt  Per_Capita  GDP_Percentage  Risk_Category
0   United States     40598.1330       93.25           21.59  Risiko Rendah
1  United Kingdom     11355.0000      278.18            5.91  Risiko Rendah
2          France      9249.4815      257.21           57.85  Risiko Rendah
3         Germany      8139.6669      149.27           46.71  Risiko Rendah
4     Netherlands      4877.3109      336.43          100.17  Risiko Tinggi
```

Penjelasan: Membuat kolom kategori risiko berdasarkan persentase PDB, menyimpan data bersih ke file CSV baru, menampilkan struktur tipe data dan lima baris pertama dari dataframe.

3. Regresi Linear

- Import Library

Kode: import pandas as pd import numpy as np import matplotlib.pyplot as plt from sklearn.model_selection import train_test_split from sklearn.linear_model import LinearRegression from sklearn.metrics import mean_squared_error, r2_score

Output: tidak ada

Penjelasan: Mengimpor pustaka python: Pandas dan NumPy untuk mengolah data, Matplotlib untuk visualisasi, Scikit-Learn untuk membagi dataset, membangun model regresi linear, dan mengevaluasi kinerja model.

- Membaca dataset hasil scraping

Kode: df = pd.read_csv("data_utang_negara_clean.csv") print(f"Jumlah baris awal: {len(df)}") print(df.head(3))

Output:
```
Jumlah baris awal: 158
         Country  External_Debt  Per_Capita  GDP_Percentage  Risk_Category
0   United States     40598.1330       93.25           21.59  Risiko Rendah
1  United Kingdom     11355.0000      278.18            5.91  Risiko Rendah
2          France      9249.4815      257.21           57.85  Risiko Rendah
```

Penjelasan: Membaca file CSV bersih, menampilkan total jumlah baris, mencetak tiga baris data pertama.

- Membuang data kosong

Kode: df = df.dropna(subset=['Per_Capita', 'GDP_Percentage', 'External_Debt'])

Output: tidak ada

Penjelasan: Menghapus baris data dari dataframe jika ada nilai kosong pada kolom Per_Capita, GDP_Percentage, maupun External_Debt.

- Menentukan nilai x dan y

Kode: X = df[['Per_Capita', 'GDP_Percentage']] y = df['External_Debt']

Output: tidak ada

Penjelasan: Memisahkan variabel independen (X) 'Per_Capita' dan 'GDP_Percentage', menentukan variabel dependen atau target (y) yaitu 'External_Debt'.

- Bagi data latih dan uji

  Kode: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0) print(f"Jumlah data latih (train): {len(X_train)}") print(f"Jumlah data uji (test): {len(X_test)}")

  Output:
  ```
  Jumlah data latih (train): 126
  Jumlah data uji (test): 32
  ```

  Penjelasan: Membagi dataset menjadi data latih dan data uji dengan pengacakan yang konsisten. kemudian kode mencetak total baris untuk masing-masing kelompok data ke layar.

- Model LinearRegression

  Kode: model = LinearRegression()

  Output: tidak ada

  Penjelasan: Menginisialisasi objek model regresi linear baru.

- Latih model dengan data latih

  Kode: model.fit(X_train, y_train) print("Model berhasil dilatih!")

  Output:
  ```
  Model berhasil dilatih!
  ```

  Penjelasan: Melatih model regresi dan mencetak pesan keberhasilan.

- Prediksi data uji

  Kode: y_pred = model.predict(X_test)

  Output: tidak ada

  Penjelasan: Menghasilkan nilai prediksi dari data uji menggunakan model.

- Evaluasi model

  Kode: koefisien = model.coef_ intercept = model.intercept_ mse = mean_squared_error(y_test, y_pred) r2 = r2_score(y_test, y_pred) print(f"Koefisien (b1, b2) : {koefisien}") print(f"Intercept (a) : {intercept}") print(f"MSE : {mse}") print(f"R2 Score : {r2}")

  Output:
  ```
  Koefisien (b1, b2) : [ 2.01343191 -1.23515697]
  Intercept (a)      : 793.878380066202
  MSE                : 914021.1887085359
  R2 Score           : -0.166469350993097
  ```

  Penjelasan: Mengekstrak parameter model (koefisien dan intersep), menghitung metrik evaluasi Mean Squared Error dan skor R-squared, kemudian menampilkan hasil.

- Persamaan Regresi

  Kode: a = intercept b1 = koefisien[0] b2 = koefisien[1] print("\n=== PERSAMAAN REGRESI ===") print("Bentuk Persamaan: Y = a + b1*X1* + *b2*X2") print(f"Y = {a:.4f} + ({b1:.4f} * Per_Capita) + ({b2:.4f} * GDP_Percentage)")

  Output:
  ```
  === PERSAMAAN REGRESI ===
  Bentuk Persamaan: Y = a + b1*X1 + b2*X2
  Y = 793.8784 + (2.0134 * Per_Capita) + (-1.2352 * GDP_Percentage)
  ```

  Penjelasan: Menyimpan nilai intersep dan koefisien ke variabel a, b1, dan b2, kemudian mencetak bentuk matematis persamaan regresi linear berganda menggunakan nilai tersebut dengan presisi empat angka desimal.

4. Decision Tree

- Import Library

  Kode: import pandas as pd import numpy as np import matplotlib.pyplot as plt from sklearn.model_selection import train_test_split from sklearn.tree import DecisionTreeClassifier from sklearn import tree from sklearn.preprocessing import LabelEncoder from sklearn.metrics import accuracy_score, classification_report

  Output: tidak ada

  Penjelasan: Mengimpor pustaka Pandas, NumPy, dan Matplotlib untuk manipulasi dan visualisasi data, Scikit-Learn untuk membagi dataset, memproses label, membangun dan mengevaluasi model klasifikasi Decision Tree.

- Membaca dataset

  Kode: df = pd.read_csv("data_utang_negara_clean.csv") print(df.head())

  Output:

	Country	External_Debt	Per_Capita	GDP_Percentage	Risk_Category
0	United States	40598.1330	93.25	21.59	Risiko Rendah
1	United Kingdom	11355.0000	278.18	5.91	Risiko Rendah
2	France	9249.4815	257.21	57.85	Risiko Rendah
3	Germany	8139.6669	149.27	46.71	Risiko Rendah
4	Netherlands	4877.3109	336.43	100.17	Risiko Tinggi

  Penjelasan: Membaca file csv kemudian menampilkan data

- Melihat informasi data

  Kode: df.info()

  Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 158 entries, 0 to 157
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Country         158 non-null    object
 1   External_Debt   158 non-null    float64
 2   Per_Capita      158 non-null    float64
 3   GDP_Percentage  158 non-null    float64
 4   Risk_Category   158 non-null    object
dtypes: float64(3), object(2)
memory usage: 6.3+ KB
```

  Penjelasan: Menampilkan ringkasan informasi dari df. Dan informasi nya yaitu jumlah baris dan kolom, nama dan tipe data, jumlah data, dan estimasi jumlah memori yang terpakai.

- Melihat deskripsi statistik

  Kode: print(df.describe())

  Output:

	External_Debt	Per_Capita	GDP_Percentage
count	158.000000	158.000000	158.000000
mean	815.058932	103.632152	68.442848
std	3547.926675	317.001242	293.296863
min	0.133000	0.600000	0.160000
25%	4.830400	21.157500	13.232500
50%	39.808750	39.860000	22.470000
75%	259.419375	84.950000	46.090000
max	40598.133000	3673.000000	3412.500000

  Penjelasan: Mencetak ringkasan statistik dari sebuah dataframe.

- Cek data kosong

  Kode: print(df.isna().sum())

  Output:

```
Country           0
External_Debt     0
Per_Capita        0
GDP_Percentage    0
Risk_Category     0
dtype: int64
```

  Penjelasan: Menampilkan jumlah nilai kosong pada setiap kolom.

- Melihat isi unik kolom target

  Kode:
  ```
  target_unik = df['Risk_Category'].unique() print("=== ISI UNIK KOLOM TARGET ===") print(f"Nilai unik target: {target_unik}")
  ```

  Komentar Penjelasan Kelas:

  1. 'Risiko Tinggi' : Negara dengan persentase utang eksternal terhadap GDP > 60%.

  2. 'Risiko Rendah' : Negara dengan persentase utang eksternal terhadap GDP <= 60%.

  Output:
  ```
  === ISI UNIK KOLOM TARGET ===
  Nilai unik target: ['Risiko Rendah' 'Risiko Tinggi']
  ```

  Penjelasan: Mengekstrak dan mencetak nilai unik pada kolom target Risk_Category untuk mengetahui pembagian kelas risikonya.

- Memisahkan fitur x dan target y

  Kode:
  ```
  x_df = df.drop(columns=['Country', 'GDP_Percentage', 'Risk_Category']) y_df = df['Risk_Category']
  ```

  Output: tidak ada

  Penjelasan: Memisahkan variabel independen (x_df) dengan menghapus kolom tertentu, dan menetapkan kolom target (y_df) untuk proses pemodelan klasifikasi.

- Mengubah x dan y menjadi array angka

  Kode:
  ```
  le = LabelEncoder() y_encoded = le.fit_transform(y_df)
  X = np.array(x_df, dtype=np.float64) y = np.array(y_encoded, dtype=np.int64)
  print("Tipe data x_array:", X.dtype) print("Tipe data y_array:", y.dtype)
  ```

  Output:
  ```
  Tipe data x_array: float64
  Tipe data y_array: int64
  ```

  Penjelasan: Mengubah kategori target menjadi angka menggunakan LabelEncoder, mengonversinya menjadi *array* NumPy, lalu mencetak tipe datanya.

- Membagi data latih dan data uji

  Kode:
  ```
  X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.3, random_state=0 ) print(f"Jumlah data latih (X_train): {X_train.shape[0]} baris") print(f"Jumlah data uji (x_test): {X_test.shape[0]} baris")
  ```

  Output:
  ```
  Jumlah data latih (X_train): 110 baris
  Jumlah data uji (x_test): 48 baris
  ```

  Penjelasan: Membagi dataset menjadi data latih dan data uji secara konsisten, lalu mencetak jumlah barisnya.

- Membuat model decision tree

  Kode:
  ```
  model_dt = DecisionTreeClassifier(random_state=0) model_dt.fit(X_train, y_train)
  ```

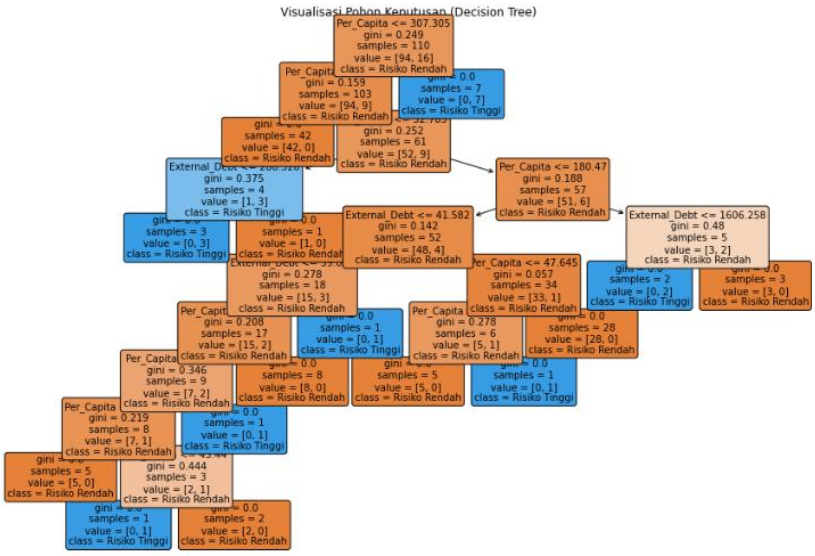
  Output:
  ```
  DecisionTreeClassifier
  DecisionTreeClassifier(random_state=0)
  ```

  Penjelasan: Menginisialisasi dan melatih model klasifikasi Decision Tree.

- Tampilan visual pohon

  Kode:
  ```
  plt.figure(figsize=(15, 10)) tree.plot_tree(model_dt, feature_names=x_df.columns, class_names=le.classes_, filled=True, rounded=True, fontsize=10) plt.title("Visualisasi Pohon Keputusan (Decision Tree)") plt.show()
  ```

Output:



Penjelasan: Membuat dan menampilkan visualisasi grafis model Decision Tree menggunakan Matplotlib, dilengkapi dengan detail nama fitur, label kelas, dan desain yang mudah dibaca.

- Menguji dengan satu data baru

Kode:
```
data_baru = np.array([[50000000000, 2000]], dtype=np.float64) prediksi_baru = model_dt.predict(data_baru) hasil_prediksi_kategori = le.inverse_transform(prediksi_baru) print("=== UJI PREDIKSI DATA BARU ===") print(f"Data input [Total Utang, Utang Per Kapita]: {data_baru}") print(f"Hasil Prediksi Kelas (Angka) : {prediksi_baru[0]}") print(f"Hasil Prediksi Kategori (Teks) : { hasil_prediksi_kategori[0]}")
```

Output:
```
=== UJI PREDIKSI DATA BARU ===
Data input [Total Utang, Utang Per Kapita]: [[5.e+10 2.e+03]]
Hasil Prediksi Kelas (Angka)    : 1
Hasil Prediksi Kategori (Teks)  : Risiko Tinggi
```

Penjelasan: Menguji data dengan satu data baru, dengan hasil akhir data baru memiliki prediksi kelas 1 atau dalam prediksi kategori yaitu Risiko Tinggi

- Hitung akurasi

Kode:
```
y_pred_dt = model_dt.predict(X_test)
akurasi = accuracy_score(y_test, y_pred_dt) print("=== EVALUASI AKURASI ===") print(f"Akurasi Model: {akurasi:.4f} ({akurasi * 100:.2f}%)")
```

Output:
```
=== EVALUASI AKURASI ===
Akurasi Model: 0.8333 (83.33%)
```

Penjelasan: Melakukan prediksi pada data uji, menghitung nilai akurasi, kemudian menampilkan hasilnya dalam format desimal dan persentase yang jelas.

- Menampilkan classification_report

Kode:
```
print("=== CLASSIFICATION REPORT ===") print(classification_report(y_test, y_pred_dt, target_names=le.classes_))
```

Output:
```
=== CLASSIFICATION REPORT ===
              precision    recall  f1-score   support

Risiko Rendah       0.88      0.93      0.90        41
Risiko Tinggi       0.40      0.29      0.33         7

     accuracy                           0.83        48
    macro avg       0.64      0.61      0.62        48
 weighted avg       0.81      0.83      0.82        48
```

Penjelasan: Mencetak judul dan menampilkan detail laporan klasifikasi evaluasi model dengan menggunakan nama kelas target yang sebenarnya.

5. Naive Bayes
   - Memanggil Library

     Kode: import pandas as pd import numpy as np import matplotlib.pyplot as plt import seaborn as sns from sklearn.preprocessing import LabelEncoder from sklearn.model_selection import train_test_split from sklearn.naive_bayes import GaussianNB from sklearn.metrics import confusion_matrix, classification_report

     Output: tidak ada

     Penjelasan: mengimpor berbagai pustaka Python untuk manipulasi data, visualisasi, serta pembuatan model pembelajaran mesin.
   - Membaca dataset dan menampilkan data

     Kode: df = pd.read_csv('final_test (NB).csv') print(df.head())

     Output:

	weight	age	height	size
0	62	28.0	172.72	XL
1	59	36.0	167.64	L
2	61	34.0	165.10	M
3	65	27.0	175.26	L
4	62	45.0	172.72	M

     Penjelasan: Membaca file CSV dan menampilkan lima baris pertamanya.
   - Melihat informasi data

     Kode: df.info()
     print("\n" + "--- DESKRIPSI STATISTIK (describe()) ---")
     print(df.describe())

     Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 510 entries, 0 to 509
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   weight  510 non-null    int64
 1   age     509 non-null    float64
 2   height  510 non-null    float64
 3   size    510 non-null    object
dtypes: float64(2), int64(1), object(1)
memory usage: 16.1+ KB

--- DESKRIPSI STATISTIK (describe()) ---
           weight         age      height
count  510.000000  509.000000  510.000000
mean    62.103922   34.182711  165.797255
std      9.766175    7.942214    6.737697
min     43.000000   17.000000  142.240000
25%     55.000000   29.000000  160.655000
50%     61.000000   32.000000  165.100000
75%     65.000000   38.000000  170.180000
max    113.000000   69.000000  187.960000
```

     Penjelasan: Menampilkan informasi struktur dari dataset, kemudian mencetak ringkasan statistik untuk kolom numeriknya.
   - Memisahkan fitur numerik

     Kode: numerical_features = df.select_dtypes(include=['number']).columns.tolist() print(f"Jumlah fitur numerik: {len(numerical_features)}") print(f"Daftar fitur numerik: {numerical_features}")

     Output:

```
Jumlah fitur numerik: 3
Daftar fitur numerik: ['weight', 'age', 'height']
```

     Penjelasan: Memilih kolom bertipe data numerik pada dataset, kemudian menampilkan jumlah total beserta daftar nama fitur numerik tersebut.
   - Memisahkan fitur kategorikal

     Kode: categorical_features = df.select_dtypes(include=['object', 'category']).columns.tolist() print(f"Jumlah fitur kategorikal: {len(categorical_features)}") print(f"Daftar fitur kategorikal: {categorical_features}")

Output:
```
Jumlah fitur kategorikal: 1
Daftar fitur kategorikal: ['size']
```

Penjelasan: Memilih semua kolom bertipe kategorikal atau objek, lalu mencetak jumlah keseluruhan beserta daftar nama fitur tersebut ke layar.

- Menampikan isi unik setiap fitur kategorikal

Kode:
```
for col in categorical_features:
    print(f"Kolom '{col}': {df[col].unique()} (Jumlah unik: {df[col].nunique()})")
```

Output:
```
Kolom 'size': ['XL' 'L' 'M' 'S' 'XXS' 'XXXL'] (Jumlah unik: 6)
```

Penjelasan: Menampilkan nilai dan jumlah unik setiap kolom kategorikal.

- Mengubah fitur kategorikal menjadi angka

Kode:
```
df_encoded = df.copy() label_encoders = {} for col in categorical_features: le = LabelEncoder() df_encoded[col] = le.fit_transform(df_encoded[col].astype(str)) label_encoders[col] = le df_encoded = df_encoded.dropna()
print(df_encoded.head())
```

Output:

	weight	age	height	size
0	62	28.0	172.72	3
1	59	36.0	167.64	0
2	61	34.0	165.10	1
3	65	27.0	175.26	0
4	62	45.0	172.72	1

Penjelasan: Menyalin dataset, mengubah fitur kategorikal menjadi angka menggunakan LabelEncoder, menyimpan setiap encoder, menghapus baris yang memiliki nilai kosong, lalu menampilkan lima baris pertamanya.

- Memisahkan fitur x dan target y

Kode:
```
x = df_encoded.drop(columns=['size'])
y = df_encoded['size']
print("Contoh 5 Baris Fitur (x):")
print(x.head())
print("\nContoh 5 Baris Target (y):")
print(y.head())
```

Output:
```
Contoh 5 Baris Fitur (x):
   weight   age  height
0      62  28.0  172.72
1      59  36.0  167.64
2      61  34.0  165.10
3      65  27.0  175.26
4      62  45.0  172.72

Contoh 5 Baris Target (y):
0    3
1    0
2    1
3    0
4    1
Name: size, dtype: int32
```

Penjelasan: Memisahkan fitur dan target dari dataset, kemudian menampilkan lima baris pertama dari keduanya.

- Membagi data uji dan data latih

Kode:
```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25, random_state=0)
print(f"Jumlah data latih (x_train): {x_train.shape[0]} baris") print(f"Jumlah data uji (x_test): {x_test.shape[0]} baris")
```

Output:
```
Jumlah data latih (x_train): 381 baris
Jumlah data uji (x_test): 128 baris
```

Penjelasan: Membagi dataset menjadi data latih dan uji, lalu menampilkan jumlah baris dari keduanya.

- Model gaussian dan latih fit

  Kode: model = GaussianNB() model.fit(x_train, y_train)

  Output:
  ```
  GaussianNB
  GaussianNB()
  ```

  Penjelasan: menginisialisasi dan melatih model algoritma Gaussian Naive Bayes.

- Prediksi data latih dan data uji

  Kode: y_pred_train = model.predict(x_train) y_pred_test = model.predict(x_test)

  Output: tidak ada

  Penjelasan: Melakukan prediksi target pada data latih dan uji.

- Evaluasi data latih

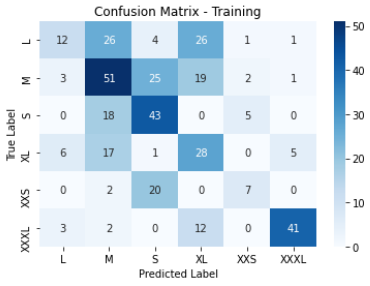
  Kode: cm_train = confusion_matrix(y_train, y_pred_train) print("Confusion Matrix (Training):\n", cm_train) print("\nClassification Report (Training):\n", classification_report(y_train, y_pred_train))
  plt.figure(figsize=(6, 4)) sns.heatmap(cm_train, annot=True, fmt='d', cmap='Blues', xticklabels=label_encoders['size'].classes_, yticklabels=label_encoders['size'].classes_) plt.title("Confusion Matrix - Training") plt.xlabel("Predicted Label") plt.ylabel("True Label") plt.show()

  ```
  Confusion Matrix (Training):
   [[12 26  4 26  1  1]
   [ 3 51 25 19  2  1]
   [ 0 18 43  0  5  0]
   [ 6 17  1 28  0  5]
   [ 0  2 20  0  7  0]
   [ 3  2  0 12  0 41]]

  Classification Report (Training):
                precision    recall  f1-score   support

             0       0.50      0.17      0.26        70
             1       0.44      0.50      0.47       101
             2       0.46      0.65      0.54        66
             3       0.33      0.49      0.39        57
             4       0.47      0.24      0.32        29
             5       0.85      0.71      0.77        58

      accuracy                           0.48       381
     macro avg       0.51      0.46      0.46       381
  weighted avg       0.50      0.48      0.47       381
  ```

  Output:

  Penjelasan: Menghitung dan mencetak matriks serta laporan klasifikasi untuk data latih, lalu memvisualisasikan hasilnya dalam bentuk grafik heatmap.

- Evaluasi data uji

  Kode: cm_test = confusion_matrix(y_test, y_pred_test) print("Confusion Matrix (Testing):\n", cm_test) print("\nClassification Report (Testing):\n", classification_report(y_test, y_pred_test))
  plt.figure(figsize=(6, 4)) sns.heatmap(cm_test, annot=True, fmt='d', cmap='Greens', xticklabels=label_encoders['size'].classes_,

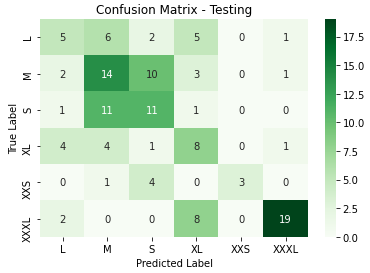
yticklabels=label_encoders['size'].classes_) plt.title("Confusion Matrix - Testing") plt.xlabel("Predicted Label") plt.ylabel("True Label") plt.show()

```
Confusion Matrix (Testing):
 [[ 5  6  2  5  0  1]
 [ 2 14 10  3  0  1]
 [ 1 11 11  1  0  0]
 [ 4  4  1  8  0  1]
 [ 0  1  4  0  3  0]
 [ 2  0  0  8  0 19]]

Classification Report (Testing):
              precision    recall  f1-score   support

           0       0.36      0.26      0.30        19
           1       0.39      0.47      0.42        30
           2       0.39      0.46      0.42        24
           3       0.32      0.44      0.37        18
           4       1.00      0.38      0.55         8
           5       0.86      0.66      0.75        29

    accuracy                           0.47       128
   macro avg       0.55      0.44      0.47       128
weighted avg       0.52      0.47      0.48       128
```



Output:

Penjelasan: Menghitung dan mencetak matriks beserta laporan klasifikasi untuk data uji, kemudian memvisualisasikan hasil evaluasi tersebut dalam bentuk grafik heatmap berwarna hijau untuk menganalisis performa akhir prediksi model.

- Uji dengan satu data baru

Kode: data_baru = pd.DataFrame([[60, 25.0, 168.0]], columns=['weight', 'age', 'height']) prediksi_angka = model.predict(data_baru) prediksi_teks = label_encoders['size'].inverse_transform(prediksi_angka) print(data_baru) print(f"Hasil Prediksi (Angka ter-encode): {prediksi_angka[0]}") print(f"Hasil Prediksi (Kategori Asli): {prediksi_teks[0]}")

```
   weight   age  height
0      60  25.0   168.0
Hasil Prediksi (Angka ter-encode): 1
Hasil Prediksi (Kategori Asli): M
```

Output:

Penjelasan: Membuat data baru, memprediksi target menggunakan model, mengubah angka prediksi menjadi teks kategori asli, lalu mencetak semua hasilnya.